

M&S: Assignment report template

Group 11

Luke Delmulle
Eduard Kuteš
Gonçalo Pinheiro
Ceyhun Ceylan

June 21, 2026

1 Introduction

We have been tasked to create a model of a 3 dimensional (3D), 3-Degree of Freedom(DOF) robot arm using cartesian impedance control with gravity compensation and cascading control loops. We will use Series Elastic Actuators(SEA) for torque control and internal current control.

We used the 3D mechanics toolbox to model the dynamics of the robot arm for each joint and link. We included the mass and inertia of the motors into the 3D model to allow for a more accurate model.

We used feedback from the 3D toolbox of the joint orientations and speeds with a forward kinematics model and the Jacobian to find speed and positions of the end effector to implement impedance control and find the desired torque at the end effectors. We also included a gravity compensation term inside the impedance controller.

Using connecting power ports; we modeled and connected the SEA elements to the 3D mechanics model in order to get feedback on the torque present in the joints by using the spring constant of the SEA and the difference in angles at the SEA to find the current torque at each joint for feedback control for the joint torques.

Based on the desired torques in the joints for our desired motion we chose motors and gearboxes for the model based on actual motors and gearboxes that we found online. These models included the motors and gearbox inertia's, reduction ratios and friction models.

Within the motor itself there is an internal feedback current controller which is used to control the current within the motor. This is necessary since the current has a direct effect on the torque the motor shall be providing.

We then tested and tuned the different controllers to ensure all of the values are realistic and work as expected. We also included external disturbances to the end effector to validate the impedance controller is working as expected.

An overview of our final model is shown below:

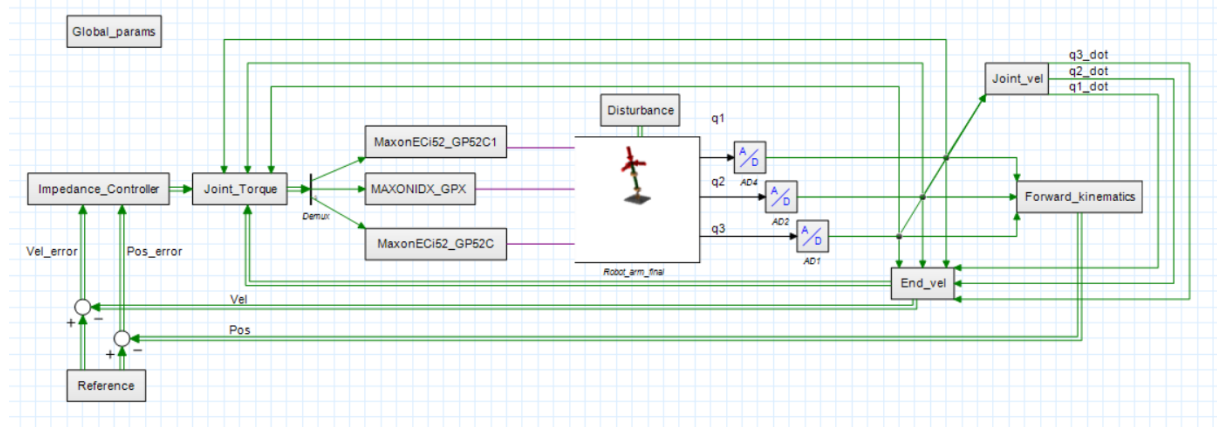


Figure 1: 20sim model overview

2 Model

We broke the model into various sections described in the introduction, namely the 3D model, Impedance controller, Torque controller, Motor model and current controller.

2.1 3D model

We used the 3D mechanics toolbox to model the robot arm since it was the simplest method to use for high complexity models.

We decide to use a yaw-pitch-pitch configuration for the robot since it was the simplest to find the forward kinematics for and it is one of the most common configurations for robot arms. We created a mathematical drawing of the configuration of the robot arm which can be viewed in Figure 2. This diagram was then used as a base model to translate to 20 sim. We also included the motors of the joints 1,2 and 3 in the base, end of link 1 and end of link 2, respectively, and small end effector. To attach a mass body (motor) to other mass body (link) we used weld joints. The final 20-sim 3D mechanics model looks as follows in 3:

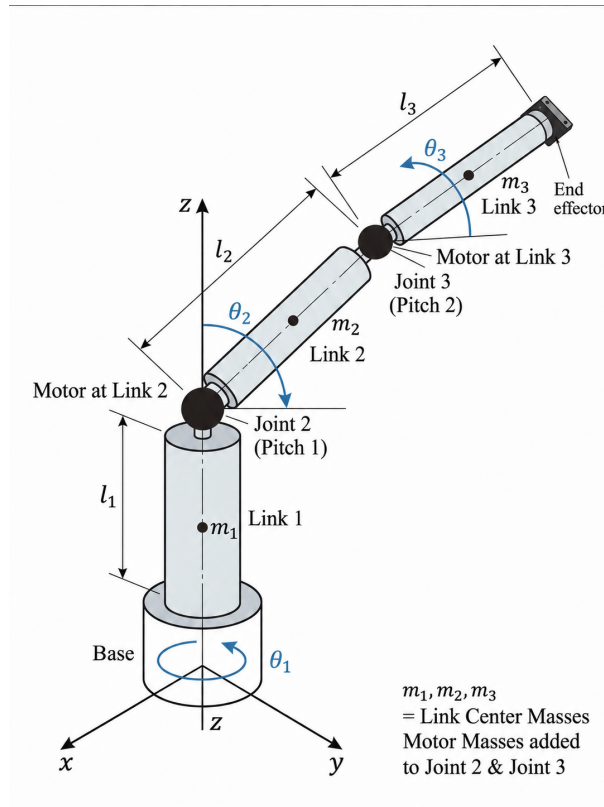


Figure 2: Diagram of robot arm

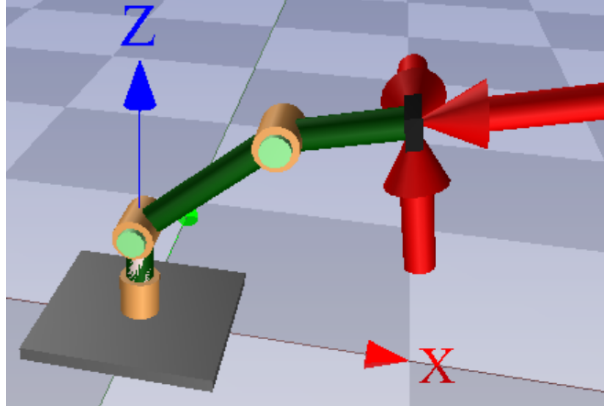


Figure 3: 3D mechanics toolbox model

When modelling each link we chose hollow cylindrical aluminium (Al 6061-T6) tubes, as this is realistic for modern lightweight robot arms. For a hollow cylinder with outer radius R_o and inner radius R_i , $I_{xx} = I_{yy}$ are the moments of inertia about the transverse axes through the centre and I_{zz} is the moment of inertia about the longitudinal (spin) axis:

$$I_{xx} = I_{yy} = m \left[\frac{R_o^2 + R_i^2}{4} + \frac{l^2}{12} \right] \quad (1)$$

$$I_{zz} = \frac{m(R_o^2 + R_i^2)}{2} \quad (2)$$

The link sections are L1 – OD 40 mm / wall 2 mm, L2 – OD 40 mm / wall 3 mm and L3 – OD 30 mm / wall 2 mm. The lengths were chosen to be realistic according to other robots we have seen and masses were calculated from the volume of the links and the density of the aluminium, which is $\rho = 2700 \text{ kg/m}^3$. The same approach was used to model the motors as cylinders. However, since the dynamic effects of their rotational moments of inertia are negligible, the motors were considered only as point masses. The masses of the selected motors were taken from their datasheets [1, 2, 3, 4, 5, 7]. Similarly, the end effector was modelled as a small block attached to the end of Link 3. As with the motors, it was simplified and represented as a point mass in the calculations. A summary of the parameters of the robot arm are shown in table 1. As it can be seen from the table 1, most of the link inertias have little to no effect since

Table 1: Physical and Inertial Parameters for the 3-DOF Robot Arm

Element	Length l (m)	Mass m (kg)	Section OD/wall (mm)	Inertia I_{xx}/I_{yy} ($\text{kg} \cdot \text{m}^2$)	Inertia I_{zz} ($\text{kg} \cdot \text{m}^2$)
Link 1	0.20	0.13	40 / 2	4.57e-04	4.71e-05
Link 2	0.60	0.57	40 / 3	0.0172	1.96e-04
Link 3	0.40	0.19	30 / 2	0.00255	3.74e-05
Motor 1 (at Joint 1)	—	1.443	—	—	—
Motor 2 (at Joint 2)	—	2.282	—	—	—
Motor 3 (at Joint 3)	—	1.443	—	—	—
End Effector	—	0.1	—	—	—

they are so small, except for Link 2's inertia I_{xx}/I_{yy} (and, to a lesser extent, Link 3's) which has a more noticeable effect.

2.2 Impedance controller

The impedance controller is built to allow the end effector of the robot to act like a spring-damper system operating between the actual position p_e and the desired position p_d it is used to calculate a desired virtual force to apply at the end effector.

2.2.1 Impedance control law equations

The virtual Cartesian force F is defined by the following control law:

$$F = K_p e + K_d \dot{e}$$

where the system kinematics and error states are governed by:

$$\begin{aligned} e &= p_d - p_e \\ \dot{p}_e &= J \cdot \dot{\theta} \\ \dot{e} &= \dot{p}_d - \dot{p}_e \end{aligned}$$

Where the individual components are defined as:

- $e \in \mathbb{R}^3$ represents the **Cartesian position error**.
- $\dot{e} \in \mathbb{R}^3$ represents the **Cartesian velocity error**.
- $\dot{p}_e$ is the **end-effector velocity**, mapped from joint velocities $\dot{\theta}$ using the geometric Jacobian matrix J .
- $K_p, K_d \in \mathbb{R}^{3 \times 3}$ are positive-definite, diagonal matrices representing the virtual **stiffness** and **damping** parameters, respectively.

We use diagonal gain matrices $K_p = 100$ N/m and $K_d = 10$ N-s/m. The stiffness was sized from the desired static compliance: at steady state the velocity error vanishes and the virtual force balances the applied load, $F = K_p e$. A 1 kg load ($F = mg = 9.81$ N) therefore produces an end-effector deflection of

$$e = \frac{F}{K_p} = \frac{9.81}{100} \approx 0.098 \text{ m} \approx 10 \text{ cm}, \quad (3)$$

which we judged realistic for a compliant end effector. The damping coefficient $K_d = 10$ N-s/m was then tuned to suppress oscillation during motion; in the final model this value produced no visible overshoot at the end effector when changing position, which is desirable.

2.2.2 Forward kinematics and Jacobian

This model allows us to find the force required at the end effector based on the desired position and velocity of the effector to the current position and velocity of the end effector. To find the current position of the end effector we need to create a forward kinematics model of the system. Using Figure 2 we find the forward kinematics model to be as follows:

$$x = \cos(q_1) (l_2 \sin(q_2) + l_3 \sin(q_2 + q_3)) \quad (4)$$

$$y = \sin(q_1) (l_2 \sin(q_2) + l_3 \sin(q_2 + q_3)) \quad (5)$$

$$z = l_1 + l_2 \cos(q_2) + l_3 \cos(q_2 + q_3) \quad (6)$$

Finding the partial derivative of the forward kinematics model with respect to each joint q_1, q_2 and q_3 we can find the Jacobian for the model. We did this using Matlab's built in Jacobian function. The Jacobian was then used to find the velocity of the end effector using the equation $\dot{x} = J(\theta)\dot{q}$. The Jacobian was calculated to be:

$$J(\theta) = \begin{bmatrix} -\sin(\theta_1)\sigma_2 & \cos(\theta_1)\sigma_1 & l_3 \cos(\theta_2 + \theta_3) \cos(\theta_1) \\ \cos(\theta_1)\sigma_2 & \sin(\theta_1)\sigma_1 & l_3 \cos(\theta_2 + \theta_3) \sin(\theta_1) \\ 0 & -\sigma_2 & -\sigma_3 \end{bmatrix} \quad (7)$$

where the intermediate kinematic terms are defined as:

$$\sigma_1 = l_2 \cos(\theta_2) + l_3 \cos(\theta_2 + \theta_3)$$

$$\sigma_3 = l_3 \sin(\theta_2 + \theta_3)$$

$$\sigma_2 = \sigma_3 + l_2 \sin(\theta_2) = l_2 \sin(\theta_2) + l_3 \sin(\theta_2 + \theta_3)$$

2.3 Torque controller

The output of the impedance controller is a force in the (x, y, z) directions, but this now needs to be converted to desired torques on the joints.

2.3.1 Joint torque with gravity compensation

The required torques at the joints can be calculated using the transverse Jacobian according to $\tau = J^T(\theta)F$. Now that we have the torques required on the joints we wanted to include a gravity compensation term to account for the mass of the robot.

The total commanding joint torque vector τ is computed by summing the impedance control torques τ_{out} and a gravity compensation vector $G(q)$:

$$\tau = \tau_{\text{out}} + G(q) \quad (8)$$

where the Impedance control forces F are mapped to the joint space via the transpose of the geometric Jacobian matrix:

$$\tau_{\text{out}} = J^T(q)F \quad (9)$$

The gravity compensation term $G(q)$ depends on the configuration of the robot and was calculated based on the robot geometry shown in Figure 2. For each link, motor, and end-effector mass, the gravitational torque about each joint was obtained as the product of the gravitational force mg and the perpendicular horizontal distance from the corresponding joint axis to the mass center of gravity. Since joint 1 rotates about the vertical axis, gravity does not create a torque around this joint. The gravity torque vector $G(q)$ is given as follows:

$$G(q) = \begin{bmatrix} 0 \\ -g_n \left[\left(\frac{m_2}{2} + m_3 + m_{\text{motor},3} + M_{\text{ee}} \right) l_2 \sin(q_2) + \left(\frac{m_3}{2} + M_{\text{ee}} \right) l_3 \sin(q_2 + q_3) \right] \\ -g_n \left[\left(\frac{m_3}{2} + M_{\text{ee}} \right) l_3 \sin(q_2 + q_3) \right] \end{bmatrix}, \quad (10)$$

where $m_{1,2,3}$ are the masses of each of the links, $m_{\text{motor},3}$ is the mass of motor 3 and M_{ee} is the mass of the end effector .

2.3.2 Internal torque controller

The desired joint torques for each joint are now calculated, but a controller is necessary to ensure the torques at the joints are as we expected. We decided to use a PD controller with a feedforward term. The feedforward term passes the desired joint torque through directly (unity gain in torque space), so the output equals the desired torque when there is no error, while the PD term corrects for any error. **Note: in the final model we then converted these torque control terms to current control for the current control loop.** In the current-control implementation the feedforward becomes $i_{ff} = 1/(K_{t,i})$, converting the desired joint torque into the equivalent motor current. $K_{t,i}$ is the product of the motor torque constant and the gearbox ratio to convert from motor current to output torque. The PD gains were tuned using the normalised series-elastic torque-control formulation in [6]. For each joint, the SEA spring of stiffness k acting on the reflected inertia $I = (J_m + J_{gb})i^2$ at the gear output forms a second-order plant whose open-loop natural frequency is

$$\omega_n = \sqrt{\frac{k}{I}}. \quad (11)$$

The closed-loop torque bandwidth is set to $\omega_0 = \alpha \omega_n$ via a tuning factor $\alpha > 1$, with critical damping $\zeta = 1$. Placing the closed-loop poles accordingly yields the normalised proportional and derivative gains

$$K_p = \alpha^2 - 1, \quad K_d = \frac{2I\omega_0\zeta}{k}. \quad (12)$$

As a worked example, for joint J_1 the reflected inertia is $I = 2.690 \times 10^{-3} \text{ kg m}^2$ (from Table 4) and the SEA stiffness is $k = 50 \text{ Nm/rad}$, so with $\alpha = 1.5$ and $\zeta = 1$:

$$\begin{aligned} \omega_n &= \sqrt{50 / 2.690 \times 10^{-3}} = 136.3 \text{ rad/s}, \\ \omega_0 &= \alpha \omega_n = 1.5 \times 136.3 = 204.5 \text{ rad/s}, \\ K_p &= \alpha^2 - 1 = 1.5^2 - 1 = 1.25, \\ K_d &= \frac{2 \times 2.690 \times 10^{-3} \times 204.5 \times 1}{50} = 0.02200 \text{ s}. \end{aligned}$$

Repeating this for every joint gives the intermediate quantities and gains in Table 2.

Joint	k (Nm/rad)	I (kg · m ²)	ω_n (rad/s)	α	ω_0 (rad/s)	K_p / K_d (s)
J_1	50	2.690×10^{-3}	136.3	1.500	204.5	1.250 / 0.02200
J_2	300	0.1237	49.3	1.150	56.6	0.3235 / 0.04668
J_3	150	2.69×10^{-3}	236	1.500	354.2	1.250 / 0.01270

Table 2: Normalised PD torque-controller tuning per joint ($\zeta = 1$, critically damped).

Because the inner loop is current-controlled, each torque-domain gain is mapped to the current domain by dividing by the torque-to-current factor $K_{t,i}$ (the same factor as the feedforward):

$$K_{p,i} = \frac{K_p}{K_{t,i}}, \quad K_{d,i} = \frac{K_d}{K_{t,i}}, \quad i_{ff} = \frac{1}{K_{t,i}}. \quad (13)$$

This yields current-domain triplets $(K_{p,i}, K_{d,i}, i_{ff})$ of approximately (1.1335, 0.01995, 0.9067) A/Nm for J_1 , (0.0622, 0.00901, 0.1929) A/Nm for J_2 , and (1.1335, 0.01152, 0.9067) A/Nm for J_3 . The torque controller is shown as follows:

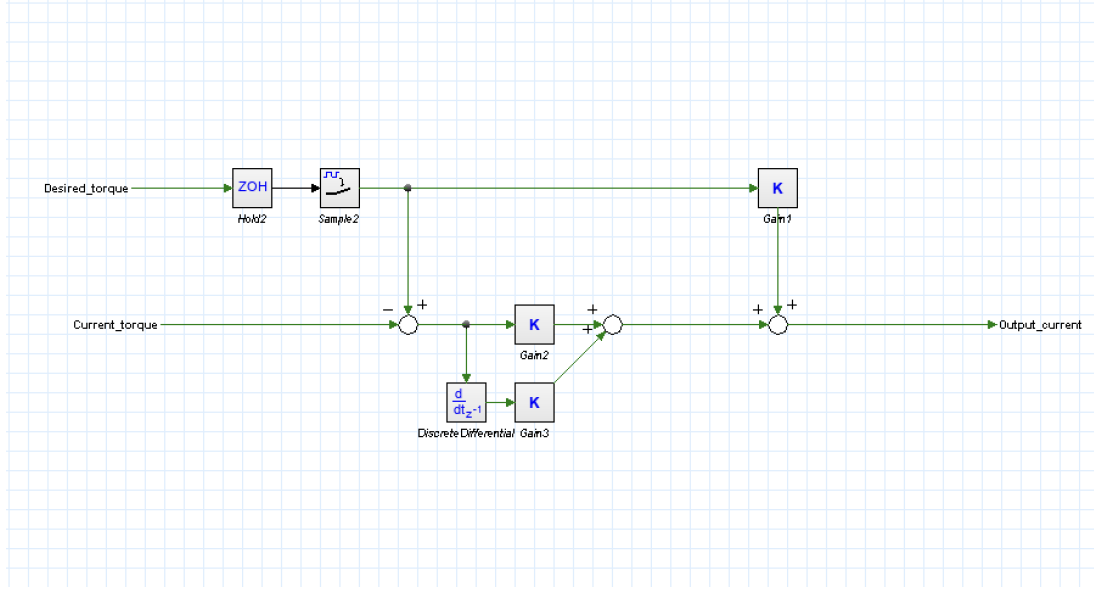


Figure 4: Torque controller

The torque controller outputs the desired current for the current controller inside the motor.

2.3.3 SEA element

In order to do torque control we need to be able to read in the current torque at each of the joints. The SEAs are used to read the torque at the joints. They act as a spring with damping, where the torque through the spring is proportional to its deflection since the spring constant of the SEA is constant. The current torque in the SEA is $\tau = k\Delta\theta$.

The model of SEAs are as follows:



Because the SEA spring sits in series between the actuator and the link, its stiffness k is a design trade-off. A *soft* spring (low k) gives more mechanical compliance, safer interaction and finer torque resolution, but it also winds up further under load and so adding a kinematic error between motor and link and it lowers the torque-loop bandwidth. A *stiff* spring (high k) does the opposite. We therefore sized each spring from a worst-case deflection limit rather than from a single nominal load.

$$k = \frac{\tau_{\text{grav}}}{\Delta\theta_{\text{target}}}, \quad \Delta\theta_{\text{target}} \approx 5^\circ = 0.087 \text{ rad.} \quad (14)$$
$$\begin{array}{ll} k_2 = 26.1/0.087 \approx 300 \text{ Nm/rad} & \Rightarrow \Delta\theta_2 = 26.1/300 \approx 5.0^\circ, \\ k_3 = 150 \text{ Nm/rad (stiffer than the 76 Nm/rad a } 5^\circ \text{ target needs)} & \Rightarrow \Delta\theta_3 = 6.6/150 \approx 2.5^\circ, \\ k_1 = 50 \text{ Nm/rad (no gravity load; softest spring)} & \Rightarrow \Delta\theta_1 \approx 2^\circ \text{ (dynamic only)}. \end{array}$$

Robustness check. As a transient worst case we also check the actuator’s *peak* (gearbox-intermittent) output torque τ_{peak} from Table 4. Even there the wind-up stays bounded $\Delta\theta = \tau_{\text{peak}}/k = 25.8^\circ$ (J_1), 12.9° (J_2) and 8.6° (J_3) comfortably within the spring’s mechanical travel, so the SEAs never over-deflect even during short torque spikes. In normal operation the joints sit at the $\approx 5^\circ$ (J_2) and $\approx 2.5^\circ$ (J_3) gravity-hold values and retain plenty of compliance headroom.

8

current loop ($\omega_{ci} = 3000$ rad/s) is feasible at every joint. The electrical and mechanical parameters of each

Table 4: Torque demand, capability and control-relevant parameters per joint.

Joint	Demand* (Nm)	Cont./Peak out (Nm)	i , gear ratio	J_{ref} (kg m ²)	τ_{el} (ms)
J_1	inertial	4.21/22.5	12	2.69×10^{-3}	0.77
J_2	26.12	45.0/67.5	62	0.124	0.50
J_3	6.59	4.21/22.5	12	2.69×10^{-3}	0.77

motor-gearbox combination, taken from the manufacturer datasheets and used throughout the friction and current-controller derivations, are summarised in Table 5.

Table 5: Electrical and mechanical parameters per joint (from datasheets).

Joint	R_m (Ω)	R_H (Ω)	R_{total} (Ω)	L (mH)	K_t (Nm/A)	J_m (kg · m ²)	J_{gb} (kg · m ²)	i	η
J_1 (Maxon EC-i 52 + GP 52 C)	0.284	0.30	0.584	0.450	0.0919	1.70e-05	1.68e-06	12	0.83
J_2 (Maxon IDX 56 L + GPX 52 UP)	0.146	0.30	0.446	0.221	0.0836	2.65e-05	5.68e-06	62	0.89
J_3 (Maxon EC-i 52 + GP 52 C)	0.284	0.30	0.584	0.450	0.0919	1.70e-05	1.68e-06	12	0.83

2.4.2 Friction

To correctly model the friction of the motor gearbox, where coulomb and viscous friction are present, we decided to use resistance block R after the gearbox TF, as can be seen in the figure 6. Because our model uses the ideal TF, e.g it contains only gearbox ratio, we are using the datasheet efficiency, maximum continuous power, maximum continuous torque and an estimate of the percentage of coulomb to viscous friction to find the parameters for our friction. We decided to use 90% coulomb friction because after some research we found that the total friction is largely dominated by coulomb friction rather than viscous friction. The friction model equation we used is as follows:

$$T_f = dC \cdot \text{sign}(\omega) + dV \cdot \omega \quad (15)$$

By building an isolated test-bench submodel and running it at nominal speed and torque, we reverse-engineered the viscous friction coefficient (dV) until the ratio of output power to input power matched the exact efficiency percentages provided by the manufacturers. Equations inside the block can be seen in listing 1.

Listing 1: Friction model parameters and equation for motor 1 in 20-sim

```
parameters
// Datasheet reference values
real eta_ref = 0.83;           // gearbox efficiency [-]
real T_ref = 4.21;             // max continuous output torque [Nm]
real P_ref = 180;              // max continuous output power [W]
// Friction loss split, input as percentages
real percC = 90;               // Coulomb friction percentage [%]
real percV = 100 - percC;      // Viscous friction percentage [%]
// tanh steepness factor (sharp sign approximation)
real w_eps = 2000;             // [s/rad]
real w_ref = P_ref / T_ref;
real T_loss_ref = T_ref * (1 / eta_ref - 1);
real fracC = percC / (percC + percV);
real fracV = percV / (percC + percV);
real dC = fracC * T_loss_ref;   // Coulomb friction torque [Nm]
real dV = fracV * T_loss_ref / w_ref; // Viscous friction coefficient [Nm/(rad/s)]
equations
p.e = dC * tanh(p.f * w_eps) + dV * p.f;
```

2.5 Current controller

The current controller needs to control the current within the motor to achieve the desired output torque. This controller needs to operate at a high bandwidth to ensure the internal controllers run faster than

the outer controllers, otherwise there could be instability. We chose a bandwidth of $\omega_{ci} = 3000$ rad/s (≈ 477 Hz) because the current controller runs at 10 kHz, and to ensure stability we needed to keep the control bandwidth well below (more than an order of magnitude lower than) the control frequency. In order to find control parameters we needed to find the dynamics for the control loop. We chose to ignore the back emf because the motors move at a slow maximum speed with the impedance controller for safety purposes. The dynamics are therefore:

$$L\dot{i} = V - (R_H + R_m)i.$$

The plant is therefore first order,

$$P(s) = \frac{I(s)}{V(s)} = \frac{1}{Ls + R}, \quad R = R_H + R_m, \quad (16)$$

with a single pole at $s = -R/L$. We close the loop with a PI controller

$$C(s) = K_p + \frac{K_I}{s} = K_p \frac{s + K_I/K_p}{s}. \quad (17)$$

The integral action removes the steady-state current error, while the PI zero is placed to cancel the plant pole:

$$\frac{K_I}{K_p} = \frac{R}{L}. \quad (18)$$

With the pole cancelled, the open loop reduces to a pure integrator, $C(s)P(s) = K_p/(Ls)$, so the closed loop is first order with bandwidth $\omega_{ci} = K_p/L$. Setting $\omega_{ci} = 3000$ rad/s and applying the pole-cancellation condition gives the two design equations

$$K_p = L \omega_{ci}, \quad (19)$$

$$K_I = R \omega_{ci}, \quad (20)$$

equivalently an integral time constant $\tau_I = K_p/K_I = L/R$ (the form implemented in the 20-sim PI block). For example, motor 1 ($L = 4.5 \times 10^{-4}$ H, $R = 0.584 \Omega$):

$$K_p = 4.5 \times 10^{-4} \times 3000 = 1.35, \quad K_I = 0.584 \times 3000 = 1752, \quad \tau_I = \frac{4.5 \times 10^{-4}}{0.584} = 7.705 \times 10^{-4} \text{ s}.$$

The final control parameters for all three motors are listed in Table 6.

Motor	K_p	K_I	$\tau_I = L/R$ (s)
1	1.35	1752	7.705×10^{-4}
2	0.663	1338	4.96×10^{-4}
3	1.35	1752	7.71×10^{-4}

Table 6: Current control parameters ($\omega_{ci} = 3000$ rad/s)

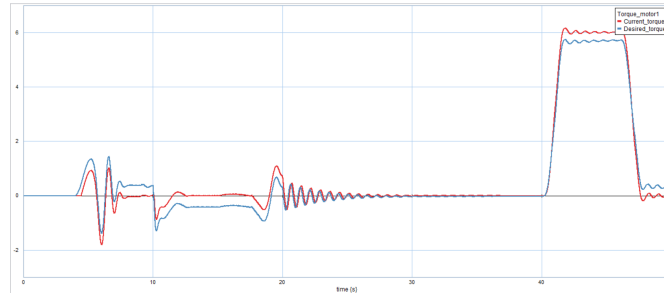
3 Simulation

When doing a simulation for our robot we set a round trajectory for the robot to follow around its central axis with the end effector facing upright. We then tested added a vertical load of $10N$ on the end effector to test the impedance controller to verify whether the end effector deflected by $0.1m$ due to the impedance controller having a stiffness of 100. We also tested the deflection in the horizontal direction for a force of $10N$.

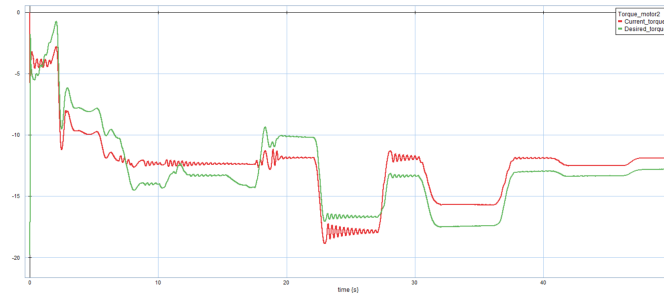
During the simulations, we realized that the difference in controller frequencies included delays and discretization. This meant that we needed to check whether this had an effect on our control parameters and meant that we had to slow down the bandwidths of the controllers to allow for slower control frequencies. We tested the torque controllers to ensure the desired torque from the controllers was within the range of the motor and gearbox minimum and maximum torques. We then tested to see whether the desired current for the motor was maintained within the were maintained within the maximum current ranges of the motor.

3.1 Torque controllers

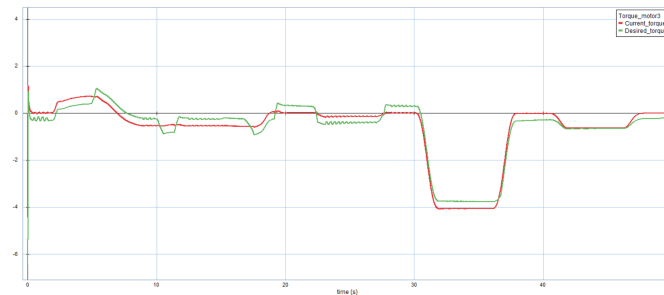
We tested the torque control inputs when the torque controller runs at a frequency of 1kHz (10 times slower than the current controller frequency). In the graphs we compare the desired and current torque for each motor when the robot is following our current trajectory which can be observed in 7. You can see the torque isn't capped for any of the motors. We also observed that there is some steady state error between the desired and current torque which is expected since the we are using a PD controller.



(a) Torque output for Motor 1.



(b) Torque output for Motor 2.

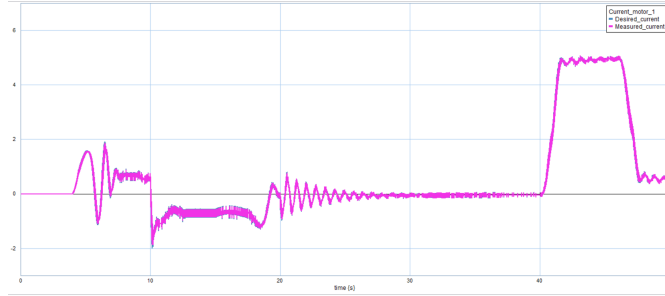


(c) Torque output for Motor 3.

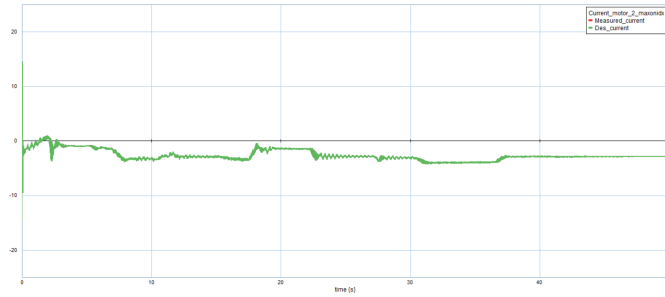
Figure 7: Simulated torque demands for the three joint motors, including gravity compensation.

3.2 Current controllers

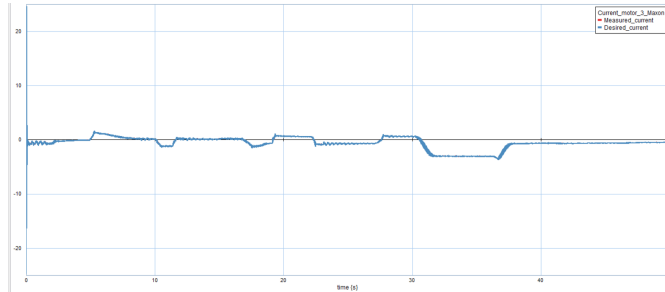
We then tested the current controllers at a frequency of 10kHz to ensure that the current limits were within the ranges of the motors. This is to ensure there is no capping of the current and that no damage occurs to the motors during operation. You can see for all of the motors the current is below their limits and the current isn't capped. A detailed simulation of the current controller is expressed in 9 to highlight the transient response of the current controller to inputs. Simulations of the desired and measured current are shown in 8.



(a) Current tracking for Motor 1 (Yaw Joint).



(b) Current tracking for Motor 2 (Lower Pitch Joint).



(c) Current tracking for Motor 3 (Upper Pitch Joint).

Figure 8: Simulated current responses for the three joint motors during trajectory execution.

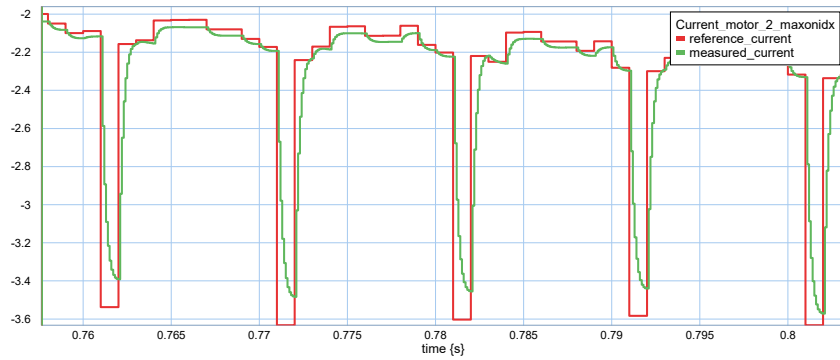
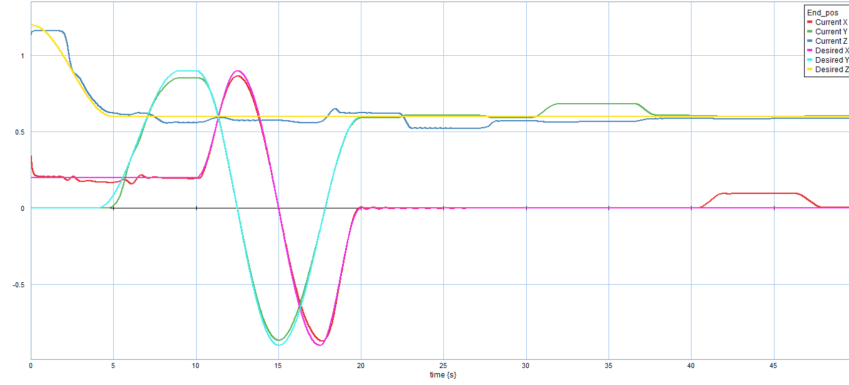


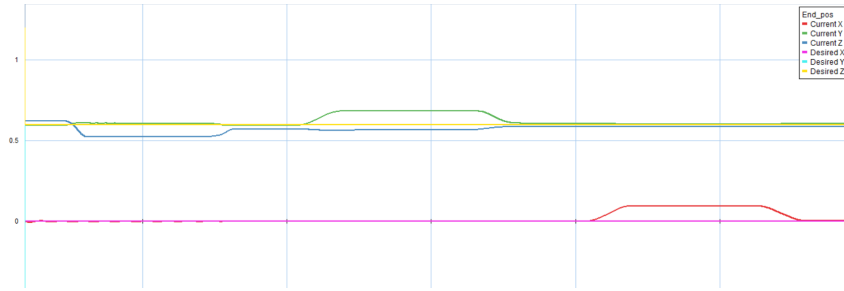
Figure 9: Detailed view of the current simulation for Motor 2, highlighting transient response.

3.3 Reference tracking with disturbance

During the simulation in we show the robot tracking a reference trajectory with the impedance controller running at 100Hz. The tracking is very accurate and follows a rotation about the z axis of the first joint. There is little steady state error which shows how the gravity compensation term is working well to compensate for the weight of the robot as well as the feed forward component for the current, but the friction within the model limits the accuracy of the feed forward term allowing for a small steady state error. The impedance controller is also working as expected . The end effector displaces around 0.1m when a 10N load is applied which is shown in 10.



(a) Overall end-effector trajectory tracking performance in Cartesian space.



(b) Displacement due to a 10N load at the effector

Figure 10: Simulated end-effector performance.

4 Discussion & Conclusion

We were able to model the robot arm quite accurately overall by including all of the main components of the simulation and including extra details to ensure the overall simulation was accurate.

We modeled the motor and gearboxes including friction models to closely represent real world motor gearbox combinations. We created 3D models of robot arms using realistic values for inertial properties, masses and motor masses attached to the robot arms. We included different discrete time domain frequencies, and encountered how the different control bandwidths are very much dependent on the discrete time domain control frequencies. We managed to control the system at 3 different control frequencies of 10kHz, 1kHz and 100 Hz and managed to maintain system stability.

We could have included an actual model of an H bridge to use rather than a Mse element which made our model potentially lose some real life accuracy. We do believe our inclusion of point masses for the motors, an end effector mass and discrete position signals and velocities calculated from differentiating the position of the end effector positions added sufficient complexity and made our model more realistic.

In conclusion we learned how to model the dynamics of complex bodies with links using different methods throughout the course. We also learned how to model physical elements into software and bond graphs with increasing accuracy. We learned to create motor and gearbox friction models which include coulomb and viscous friction using maximum continuous power and maximum continuous torques from the data sheets of the motors and gearboxes. One of the most valuable things we learned was cascaded control for complex systems including gravity compensation, Jacobian torque control, impedance control and how discretization affects control parameters. We also learnt to use modeling and simulations from a top down approach to help make decisions about physical components such as motor choices.

References

- [1] maxon Group. *GPX 52 UP Ultra Performance Planetary Gearhead — Data*, 2020. Catalog pp. 354–355, April 2020 edition. 3-stage, 62:1; continuous/intermittent output 45/67.5 Nm, $\eta = 89\%$.
- [2] maxon Group. *IDX 56 L Integrated Brushless DC Drive — Motor Data, 48 V winding*, 2020. Catalog pp. 221–222, April 2020 edition. $K_t = 83.6$ mNm/A, $R = 0.148$ Ω , $L = 0.221$ mH, $J_m = 265$ gcm².
- [3] maxon Group. *EC-i 40 130 W Brushless DC Motor (#666602, 48 V) — Data*, 2025. Catalog p. 311. $K_t = 97.8$ mNm/A, $R = 0.862$ Ω , $L = 0.922$ mH, $J_m = 78$ gcm², nominal torque 343 mNm.
- [4] Nanotec Electronic GmbH & Co. KG. *GP56-N1-3-SR Low-Noise Planetary Gearbox — Data Sheet*, 2021. Reduction ratio 3.243:1, $\eta = 93\%$, $J_{gb} = 5.5 \times 10^{-6}$ kg m², rated output 6.09 Nm.
- [5] Nanotec Electronic GmbH & Co. KG. *DB56D036030-A Brushless DC Motor — Data Sheet*, 2022. Rev. 02. Rated/peak torque 0.6/1.8 Nm, $K_t = 0.08$ Nm/A, $R = 0.53$ Ω , $L = 0.58$ mH, $J_m = 4.6 \times 10^{-5}$ kg m².
- [6] Wesley Roozing, Jörn Malzahn, Navvab Kashiri, Darwin G. Caldwell, and Nikos G. Tsagarakis. On the stiffness selection for torque controlled series-elastic actuators. *IEEE Robotics and Automation Letters*, 2(4):2255–2262, 2017.
- [7] Mark W. Spong, Seth Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. John Wiley & Sons, New York, 2006.